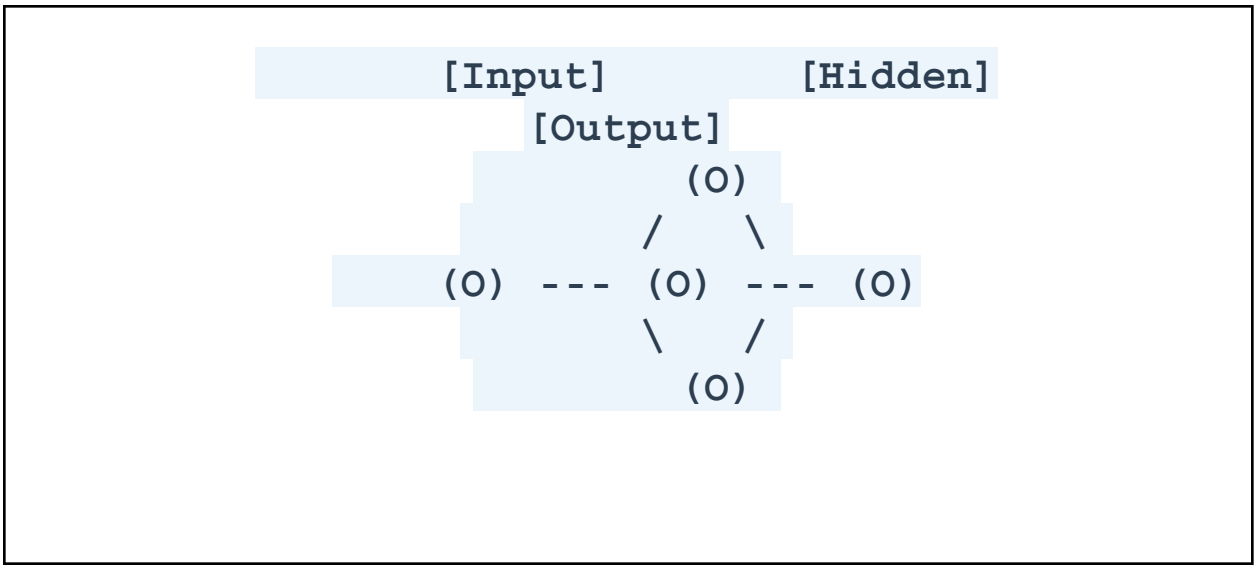
PAGE 1 — COVER

Fundamentals of Neural Networks

Neurons • Neural Network Terminology • Activation Functions



Student Name:	Mohamed Asim M
Course:	BSc Artificial Intelligence
Department:	Computer Science
Institution:	Periyar Maniammai Institute of Science & Technology
Academic Year:	2026–2027

PAGE 2 — FROM AI TO NEURAL NETWORKS

The AI Hierarchy

Field	Description
Artificial Intelligence (AI)	The overarching science of creating machines capable of mimicking human intelligence and reasoning.
↪ **Machine Learning (ML)**	A subset of AI where systems learn from data to improve performance without being explicitly programmed.

Field	Description
↪ **Deep Learning (DL)**	A subset of ML that uses multi-layered mathematical structures to extract high-level features from raw data.
↪ **Neural Networks (NN)**	The core architecture behind Deep Learning, inspired by biological brains, utilizing interconnected nodes to process complex information.

Why Machines Need Models

Machines cannot "understand" data natively. They need mathematical models to discover hidden patterns, map inputs to outputs, and make generalizations about unseen data. Neural networks became crucial because they scale incredibly well with large amounts of data and compute, solving non-linear problems that traditional algorithms struggle with.

Common Applications

- 📸 **Image Recognition:** Identifying objects, faces, and medical anomalies in images.
- 🎙️ **Speech Recognition:** Converting spoken language into text (e.g., virtual assistants).
- 📈 **Recommendation Systems:** Suggesting products, movies, or content based on user behavior.
- 📖 **Natural Language Processing (NLP):** Understanding, translating, and generating human language.
- 👁️ **Computer Vision:** Enabling self-driving cars and robotics to perceive their environment.

PAGE 3 — UNDERSTANDING THE

NEURON

The Biological Inspiration

The biological neuron is the fundamental unit of the human brain. It receives chemical signals through dendrites, processes them in the cell body, and fires an electrical impulse down the axon to communicate with other neurons via synapses.

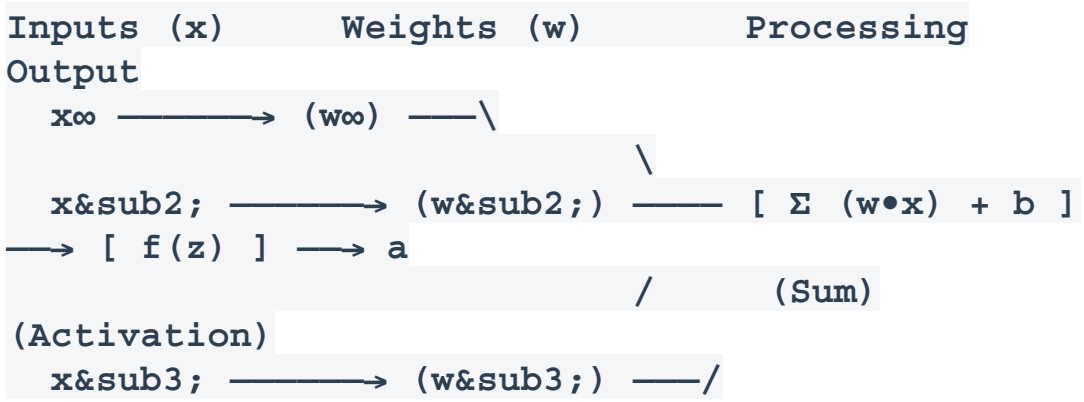
Biological Neuron	Artificial Neuron
• **Dendrites:** Receive signals from other neurons. • **Synapses:** Determine the strength of the incoming signal. • **Cell Body (Soma):** Accumulates incoming signals. • **Axon:** Transmits the signal to the next neuron if the threshold is met.	• **Inputs (x):** Receive data features or outputs from previous layers. • **Weights (w):** Determine the importance of the input. • **Summation (Σ):** Calculates the weighted sum of inputs plus bias. • **Activation Function:** Determines the final output passed to the next layer.

KEY TAKEAWAY:

An artificial neuron is a mathematical model inspired by the way biological neurons process signals. It does not perfectly replicate human biology, but it borrows the concept of weighted inputs triggering a specific output.

PAGE 4 — INSIDE AN ARTIFICIAL NEURON

The Anatomy of Computation



The Core Equations

$$z = \Sigma w_i x_i + b$$

$$a = f(z)$$

Inputs (x) & Weights (w)	Weighted Sum (z) & Bias (b)	Activation & Output (a)

Data enters as inputs. Each input is multiplied by a weight, which scales the input based on its importance.	The neuron sums all weighted inputs. A bias (b) is added to shift the sum, allowing the model to fit data better.	The result (z) is passed through an activation function f(z) to introduce non-linearity, producing the final output (a).

Numerical Example:

Suppose Inputs = [2, 3], Weights = [0.5, -1.0], Bias = 1

z = (2 * 0.5) + (3 * -1.0) + 1 = 1 - 3 + 1 = -1

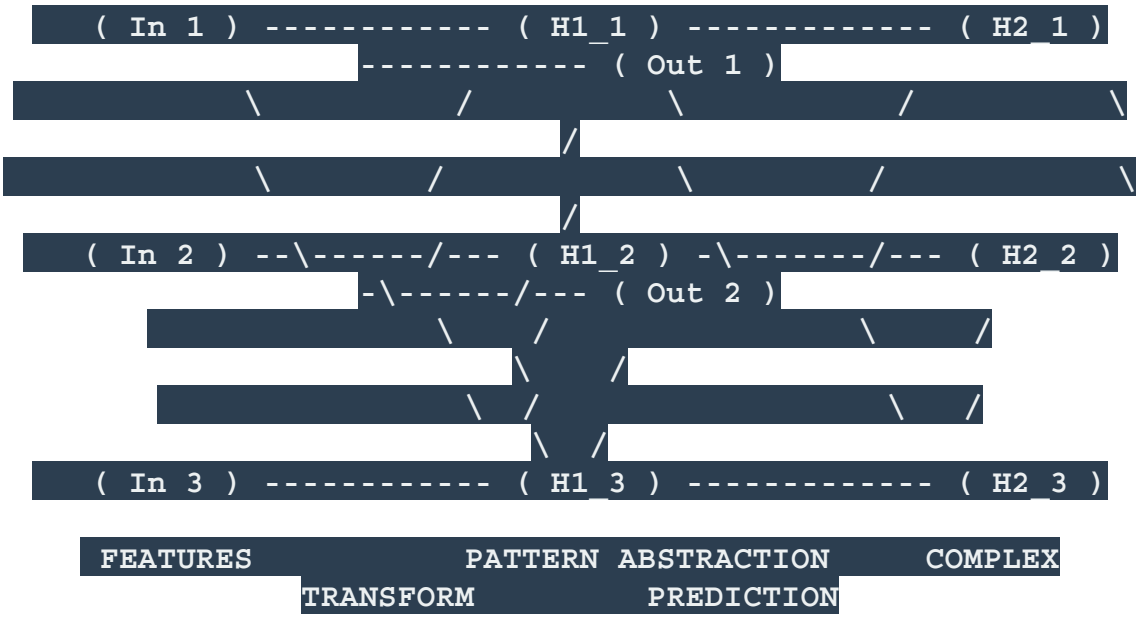
If f(z) is ReLU (which turns negatives to 0): **Output a = 0.**

PAGE 5 — BUILDING A NEURAL NETWORK

From Neurons to Layers

Individual neurons are powerful, but their true potential is unlocked when connected in vast arrays called Neural Networks.

```
[INPUT LAYER]        [HIDDEN LAYER 1]        [HIDDEN LAYER 2]
                                    [OUTPUT LAYER]
```



Input Layer

Receives the raw features from the dataset. The number of neurons equals the number of features. It performs no computation.

Hidden Layers

Reside between the input and output. They transform information and extract complex patterns. Deep learning refers to networks with multiple hidden layers.

Output Layer

Produces the final prediction. Its structure depends on the task (e.g., one neuron for binary classification, multiple neurons for multi-class).

Forward Propagation

INPUT → PROCESSING → PREDICTION

Information flows strictly in one direction from input to output, computing the mathematical transformations layer by layer.

PAGE 6 — NEURAL NETWORK VOCABULARY

The Dictionary of Deep Learning

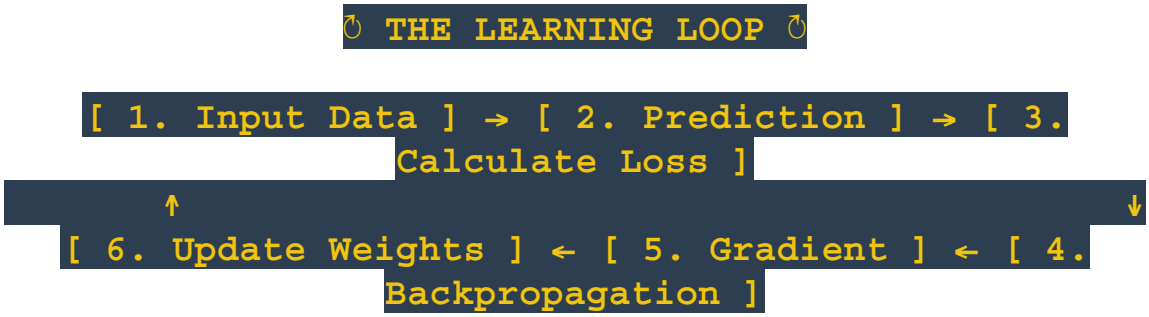
Term	Definition
Neuron / Node	The basic computational unit that takes inputs, calculates a weighted sum, and applies an activation function.
Input / Feature	A measurable property or characteristic of the data fed into the network.
Weight	A learnable parameter that determines the strength/importance of the connection between two neurons.
Bias	An additional learnable parameter added to the weighted sum to shift the activation function.

Term	Definition
Parameter	Internal variables (Weights and Biases) that the model learns and updates during training.
Activation	The mathematical function applied to a neuron's output to introduce non-linearity.
Layer	A collection of neurons operating at the same depth within the network structure.
Input / Hidden / Output Layer	Input receives data; Hidden extracts patterns; Output delivers the final result.
Connection	The link passing data from a neuron in one layer to a neuron in the next.
Architecture	The overall structural design of the network (number of layers, types of nodes, connections).
Forward Propagation	The process of passing input data through the network to calculate a prediction.
Prediction	The final output guess made by the network after forward propagation.

PAGE 7 — HOW NEURAL NETWORKS LEARN

The Training Cycle

Training a neural network is an iterative process of making predictions, measuring errors, and adjusting weights to improve future predictions.



Dataset: The complete collection of data used for training and evaluating the model.	**Ground Truth:** The actual, correct answer in the dataset.
Training / Val / Test Sets: Data split for learning, tuning, and final evaluation.	**Loss / Cost:** The mathematical measure of how wrong the network's prediction is.
Backpropagation: The algorithm that calculates the error contribution of each weight backwards through the network.	**Gradient:** The direction and rate of change of the loss concerning the weights.
Optimizer: The algorithm (e.g., Adam, SGD) used to update the weights based on	**Learning Rate:** The step size taken when updating weights.

the gradient.	
Epoch: One complete pass through the entire training dataset.	**Batch / Iteration:** Training on a small chunk of data before updating weights.

Crucial Distinction:

• **Parameter:** Variables strictly learned by the network during training (Weights, Biases).

• **Hyperparameter:** Settings chosen by the developer *before* training (Learning Rate, Number of Epochs, Batch Size, Network Depth).

PAGE 8 — ACTIVATION FUNCTIONS: THE NON-LINEARITY

Breaking the Linear Constraint

What is an activation function?

It is a mathematical "gate" applied at the end of a neuron (a = f(z)) that dictates whether, and to what extent, the neuron's signal should progress to the next layer.

Why do we need it? / What happens without it?

Without activation functions, neural networks would only perform linear operations (multiplying and adding). No matter how many layers a network has, a stack of linear operations mathematically collapses into a single linear operation. The network would be incapable of learning complex, real-world patterns like curves, circles, or intricate boundaries.

Linear Operations Only	VS	Linear + Non-Linear Activation
Limited Ability Can only draw straight lines through data. Fails at complex modeling (like identifying objects in images or understanding speech).	→	**Universal Approximator** Can model highly complex, curved, and intricate relationships. Allows the network to learn virtually any function.

Real-World Analogy: Imagine trying to map the curving borders of a country using only a single, perfectly straight ruler. It's impossible. Non-linear activation functions bend the ruler, allowing the network to wrap around complex data boundaries. Different functions transform the weighted sum (z) in uniquely beneficial ways.

PAGE 9 — CLASSIC ACTIVATION FUNCTIONS

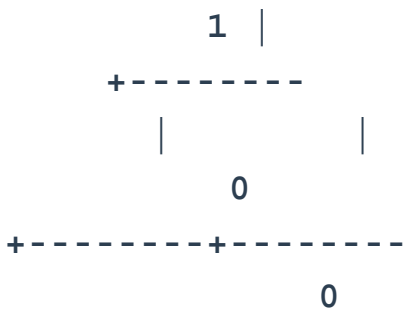
1. Binary Step

Formula: f(x) = 1 if x ≥ 0, else 0

Range: {0, 1}

Characteristics: A strict threshold. Outputs a definitive yes or no.

Typical Usage: Very early theoretical networks (Perceptrons). Rarely used today because its gradient is zero, stalling backpropagation.



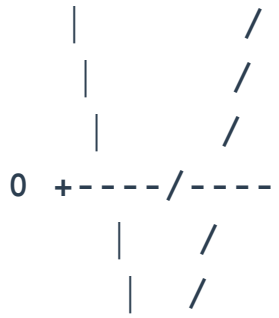
2. Linear (Identity)

Formula: f(x) = x

Range: (-∞, +∞)

Characteristics: Passes the output directly without change.

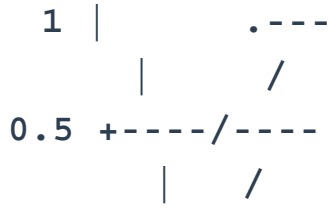
Typical Usage: Only used in the output layer of Regression models (predicting continuous values like price or temperature).

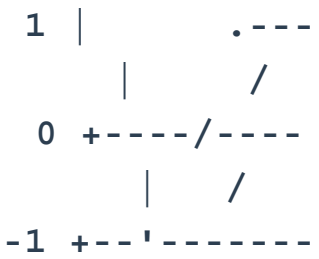


3. Sigmoid (Logistic)

Formula: f(x) = 1 / (1 + e^{-x})

Range: (0, 1)



Advantages: Smooth gradient, maps values to probabilities. **Limitations:** Suffers from "vanishing gradients" for high/low inputs; not zero-centered. **Typical Usage:** Output layer of Binary Classification.	
### 4. Tanh (Hyperbolic Tangent) **Formula:** $f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$ **Range:** (-1, 1) **Advantages:** Zero-centered, making optimization easier than Sigmoid. **Limitations:** Still suffers from the vanishing gradient problem. **Typical Usage:** Hidden layers in traditional RNNs.	

PAGE 10 — MODERN ACTIVATION FUNCTIONS

⭐ **ReLU is widely used in hidden layers because of its simplicity and computational efficiency, solving many vanishing gradient problems.**

ReLU (Rectified Linear Unit)

Formula: f(x) = max(0, x)

Range: [0, ∞)

Advantages: Computationally cheap, accelerates convergence, avoids vanishing gradient for positive values.

Limitations: "Dying ReLU" problem (negative inputs become exactly zero and neurons die).

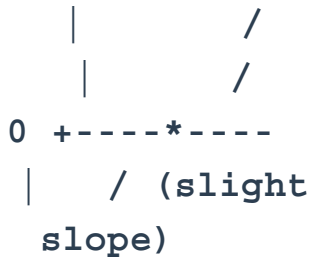
Usage: Default for Hidden Layers in CNNs and standard Deep Networks.

Leaky ReLU

Formula: f(x) = x if x > 0, else αx (where α is small, e.g., 0.01)

Advantages: Solves the Dying ReLU problem by allowing a small, non-zero gradient for negative values.

Limitations: α is a hyperparameter you must



guess.

PReLU & ELU

- **PReLU (Parametric ReLU):** Similar to Leaky ReLU, but α is a *parameter learned by the network* during training.
- **ELU (Exponential Linear Unit):** f(x) = x if x > 0, else α(e^x - 1). Smooths the curve for negative values, yielding faster convergence and better robustness, but is computationally more expensive.

GELU (Gaussian Error Linear Unit)

Formula: f(x) = x * Φ(x) (Weights inputs by their percentile).

Intuitive Explanation: Merges the idea of dropout (random zeroing) and ReLU. It smoothly gates values based on a Gaussian distribution.

Usage: Standard in massive modern architectures like Transformers (BERT, GPT).

PAGE 11 — SOFTMAX + ACTIVATION FUNCTION SELECTION

Softmax: The Multi-Class Output

Softmax is a special activation function used almost exclusively in the output layer for multi-class classification tasks. It calculates the relative probabilities of mutually exclusive classes.

Formula: f(x□) = e^(x□) / Σ e^(xΘ)

Key Properties:

- Converts raw network scores (logits) into probabilities.
- Each output strictly lies between 0.0 and 1.0.
- All outputs in the layer add up to exactly 1.0 (100%).

Example Output

Class	Probability
Cat	0.10 (10%)
Dog	**0.75 (75%)**
Bird	0.15 (15%)

The Decision Guide: Which to Choose?

Network Region	Task Type	Recommended Activation
Output Layer	Regression (Predicting a number)	**Linear**
	Binary Classification (Yes/No, 0/1)	**Sigmoid**

Network Region	Task Type	Recommended Activation
	Multi-class Classification (A, B, or C)	**Softmax**
Hidden Layers	Pattern Extraction / General compute	**Commonly ReLU / Variants**

Comparison Summary

Activation	Range	Common Use	Main Strength	Main Problem
Sigmoid	(0, 1)	Binary Output	Probability mapping	Vanishing gradient
Tanh	(-1, 1)	RNN Hidden	Zero-centered	Vanishing gradient
ReLU	[0, ∞)	CNN/MLP Hidden	Fast, prevents vanishing	Dying ReLU
Leaky ReLU	(-∞, ∞)	Deep Hidden	Prevents dying nodes	Hyperparameter tuning
GELU	(-∞, ∞)	Transformers	Smooth gating	Complex math
Softmax	(0, 1) sums to 1	Multi-class Out	Exclusive probabilities	Requires specific loss

PAGE 12 — COMPLETE NEURAL NETWORK SUMMARY

Features → Input Layer → Weighted Sum + Bias → Activation Function → Hidden Layers → Output → Loss → Backpropagation → Weight Updates → Learning

Key Concepts to Remember

- **Neuron Mechanics:** A neuron receives inputs, scales them by weights, adds bias, and produces an output.
- **Parameter Roles:** Weights determine the importance of inputs; Bias shifts the neuron's activation baseline.
- **Structure:** Layers organize neurons into a network (Input → Hidden → Output).
- **Non-linearity:** Activation functions introduce non-linearity, allowing the network to model complex real-world data.
- **Error Measurement:** Loss functions measure the deviation of predictions from the ground truth.
- **Credit Assignment:** Backpropagation calculates how parameters should change to minimize error using gradients.
- **Optimization:** Optimizers (like SGD or Adam) update the parameters based on the gradients.
- **Training Time:** Epochs represent complete passes through the training data.
- **Output Formatting:** The output activation strictly depends on the problem being solved (Regression vs Classification).

Conclusion:

Understanding neurons, neural-network terminology, and activation functions provides the foundational knowledge required to study deeper topics such as backpropagation, gradient descent, CNNs, RNNs, transformers, and the broader field of deep learning.

Next Topics to Learn →

Gradient Descent	→	Backpropagation	→	CNNs	→	RNNs	→	Transformers	→	Deep Learning